Experiment 32: Implement Pattern Matching

Aim

To write and execute a **Prolog program to implement pattern matching**.

Objectives

1. To understand pattern matching in Prolog.
2. To learn how Prolog compares structures and lists.
3. To understand unification between terms.
4. To implement pattern matching using Prolog predicates.
5. To verify whether two given patterns match.

Algorithm

1. Start the program.
2. Define a predicate pattern_match/2.
3. Compare the two given terms.
4. If both terms are identical, return true.
5. If the terms are lists, compare their heads and tails recursively.
6. Continue until all elements are matched.
7. If any element does not match, return false.
8. Stop.

Flowchart

```
      START
        |
        v
  Read Two Patterns
        |
        v
 Compare the Patterns
        |
        v
   Are They Equal?
    /       \
   Yes       No
   |         |
```

```
    v           v
  Match      Are They Lists?
               /    \
             Yes      No
              |        |
              v        v
        Compare Head &   No Match
            Tail           |
              |            |
              v            v
          Recursive      STOP
            Match
              |
              v
             STOP
```

Program

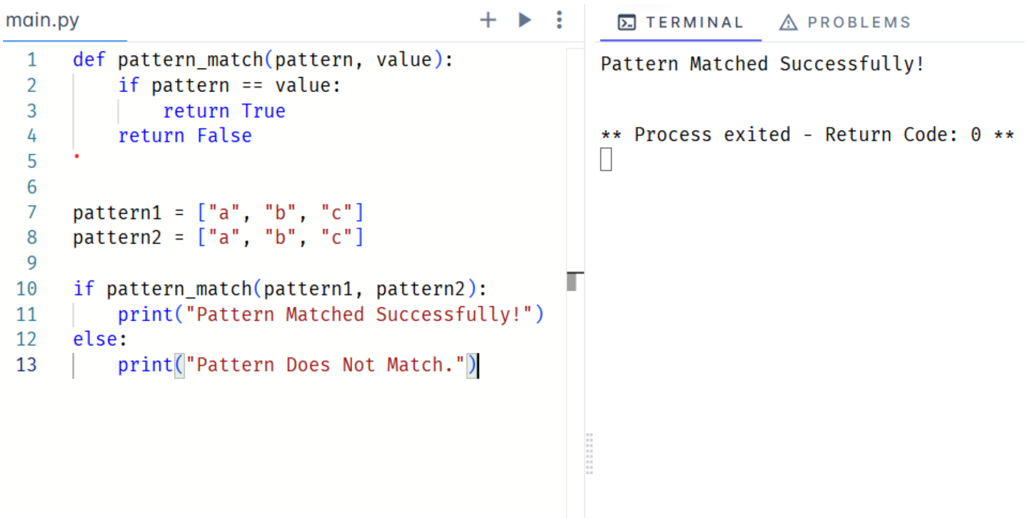
```
% Pattern Matching Program

pattern_match(X, X).

pattern_match([], []).

pattern_match([H1|T1], [H2|T2]) :-
    pattern_match(H1, H2),
    pattern_match(T1, T2).

% Test Cases

test :-
    pattern_match([a,b,c], [a,b,c]),
    write('Pattern Matched Successfully'), nl.
```

test :-

\+ pattern_match([a,b,c], [a,b,d]),

write('Pattern Does Not Match'), nl.

Output:



Result

The Prolog program was successfully executed to implement **pattern matching**.

Conclusion

pattern matching was successfully implemented using Prolog. The experiment demonstrates the use of **unification, recursion, and list processing** in Prolog.